

TD n°3

Matière : Introduction au Big Data

Classe : 1ère année MPSD

Enseignant TD : Mohamed Anouar DAHDEH



Objectif : Maîtriser l'algorithme **MapReduce** à travers des études de cas

Etude du premier cas - Analyse statique d'un texte :

L'une des premières applications de l'algorithme **MapReduce** pour un moteur de recherche comme google a été l'analyse de textes en vue de construire des annuaires inversés (quelles pages contiennent quels mots clés) et de statistiques de toutes sortes. Considérons un exemple dans cette deuxième catégorie et imaginons que nous souhaitons analyser plusieurs millions de documents et construire l'histogramme de la longueur des mots qu'ils contiennent. Admettons que nous nous intéressions à trois catégories de mots, classés selon leur longueur.

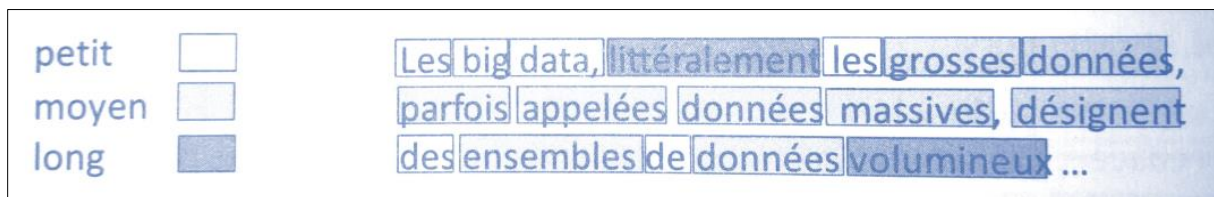


Figure 1.1- Un texte dont les mots ont été triés en trois catégories.

Les mots longs, de plus de 10 lettres, les mots de longueur moyenne de de 5 à 9 lettres et les petits mots de moins de cinq lettres

Avant de transmettre la liste de fichiers aux **mappers**, le framework la scindera en lots plus petits, chacun étant confié à l'un des **mappers**. Dans le cas de figure où l'on dispose d'un millier de **mappers**, chaque mapper aura donc à traiter quelques milliers de fichiers. On supposera pour simplifier que chaque fichier individuel est suffisamment court pour être traité par une opération **map**. Le rôle des fonctions **map** et **reduce** est dès lors facile à concevoir.

La fonction **map**, rappelons-le, prend en entrée un couple de clé –valeur. La clé en l'occurrence n'est autre que le nom ou l'identifiant d'un document et la valeur de son contenu. La fonction **map** effectuera le travail de décompte des trois catégories de mots au niveau d'un document. Chaque invocation de **map** fournira donc une liste de fréquence des 3 catégories de mots dans le document, c'est la liste de clé-valeur intermédiaire.

Le framework fusionnera ensuite ces listes intermédiaires, les triera et les scindera par mot pour construire de nouveaux lots, chacun étant assigné à un **reducer**.

La fonction **reduce**, rappelons-le, prend en entrée une liste de valeurs intermédiaires associées à une même clé et calcule l'agrégat souhaité. Chaque invocation de **reduce** calculera en l'occurrence la somme des décomptes partiels pour une des 3 catégories des mots.

Travail demandé :

Nous souhaitons calculer l'histogramme de fréquence de mots d'une certaine longueur (petit, moyen et long) pour les deux fichiers illustré par la figure 1.2 et la figure 1.3:

- 1- Déterminer le nombre de **mappers**
- 2- Donner le résultat de la fonction **map**
- 3- Déterminer les listes d'enregistrement intermédiaires résultat du **shuffle**
- 4- Déduire le nombre de **reducers**
- 5- Donner le résultat de la fonction **reduce** produite après **shuffle**

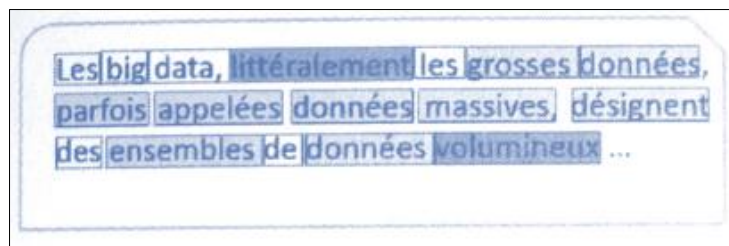


Figure 1.2 – Fichier 1

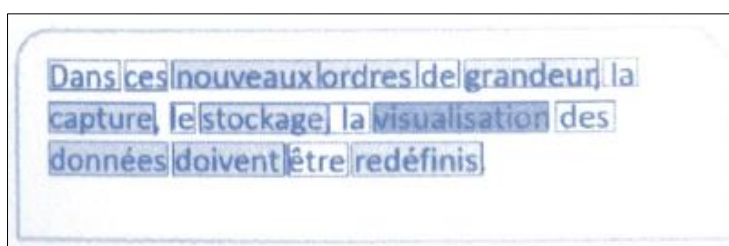


Figure 1.3 – Fichier 2

Etude du deuxième cas – Calcul d'une jointure entre deux tables:

Le calcul d'une jointure entre deux tables d'un modèle relationnel est opération particulièrement gourmande en ressources. A tel point que certains modèles de données en agrégats utilisés par certains systèmes NoSQL ont été conçus pour les éviter complètement dans un Big data. On préfère dans ces situations colocaliser les données qui sont reliées dans un

seul et même enregistrement. Dans certains cas cependant l'algorithme MapReduce vient alors à la rescousse à point nommé.

Rappelons qu'une jointure entre deux tables A et B, envisagé comme des listes d'enregistrement, est définie par deux clés, une par table. Le résultat sera une nouvelle relation dont les enregistrements sont obtenus en concaténant tous les couples formés d'un enregistrement de A avec un enregistrement de B dont les deux clés coïncident (figure 2.1).

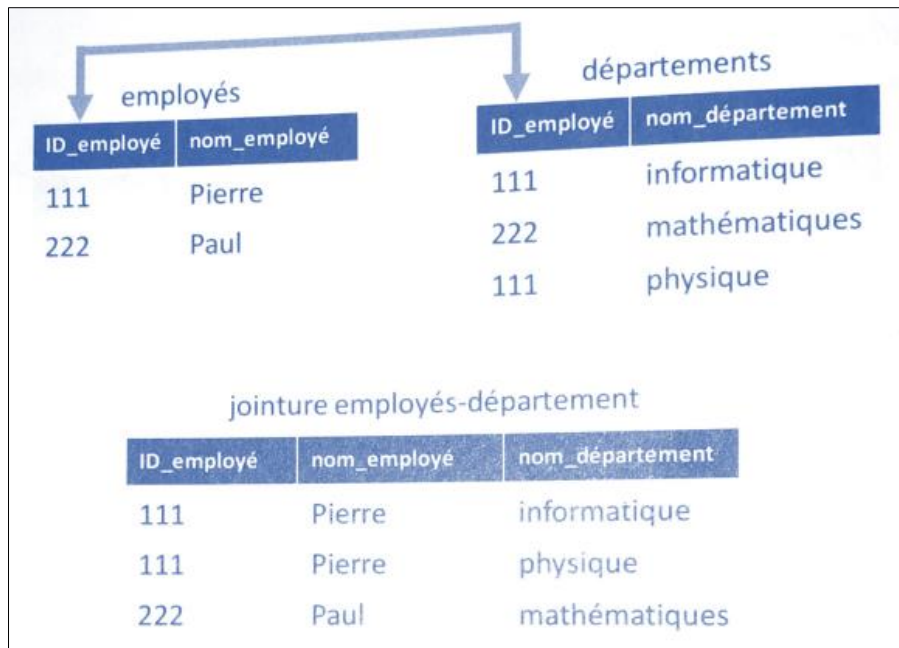


Figure 2.1- Jointure entre deux tables (employés et département), définie par un couple de clés

L'idée de base pour réaliser cette jointure avec **MapReduce** consiste à réaliser une opération en apparence triviale avant même la phase de **map**, à savoir regrouper les enregistrements des deux tables dans une seule longue liste d'enregistrement. De plus, pour chaque enregistrement, on retiendra non seulement son contenu, mais également le nom de la table dont il est issu (figure 2.2). Bien que logiquement séparée du reste du traitement, cette tâche pourra en fait être prise en charge par les fonctions **map** elles-mêmes.

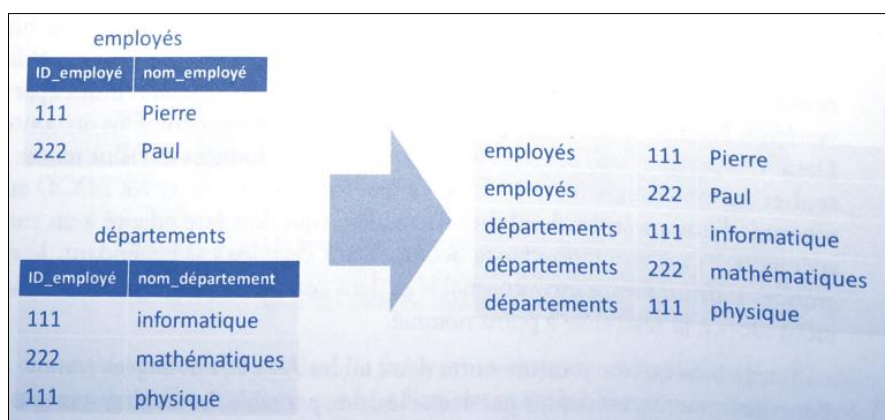


Figure 2.2- Regroupement des enregistrements dans une liste pour la fonction Map

La tâche d'une opération **map** est dès lors d'une simplicité enfantine : pour chaque enregistrement en entrée, elle renverra un couple clé-valeur dont la clé est la clé de jointure et la valeur est le contenu de l'enregistrement auquel on a adjoint le nom de la table originale.

Le framework regroupera ensuite en listes d'enregistrements intermédiaires les enregistrements qui partagent une même clé de jointure. Chacune de ces listes est affectée ensuite à un **reducer**. Une invocation d'opération **reduce** a pour tâche de produire la jointure proprement dite, c'est-à-dire d'effectuer toutes les concaténations possibles d'un enregistrement de la table A avec un enregistrement de la table B associées à une même valeur de clé de jointure

Travail demandé :

En se basant sur l'exemple de jointure de la table employés et département :

- 1- Donner le résultat de la fonction **map** qui retourne le contenu d'un enregistrement et le nom de la table originale dans la valeur de retour. La clé est la clé de la jointure.
- 2- Donner les listes d'enregistrement intermédiaires résultat du **shuffle**
- 3- Donner le résultat de la fonction **reduce** qui produit toutes les concaténations voulues à partir d'une liste d'enregistrement intermédiaires résultat du **shuffle**. Pour chaque valeur de clé de jointure, elle produit la liste des concaténations des couples d'enregistrements associés à cette clé